2026 ASU International Conference in Emerging Technologies for Sustainability and Intelligent Systems (ICETSIS) | 979-8-3315-7229-7/26/$31.00 ©2026 IEEE | DOI: 10.1109/ICETSIS68266.2026.11549348

A Multi-Scale Monetary Precision Framework for Distributed Banking Systems

Bhaskar Manchuri
Independent Researcher
McKinney, USA
bhasman@ieee.org

***Abstract*—Modern banking systems simultaneously maintain monetary values across computation (8–10 decimal places), ledger posting (4–8 decimal places), and customer presentation (2 decimal places) scales. In distributed microservice and event-driven architectures, unmanaged transitions between these scales introduce rounding drift, ledger divergence, reconciliation breaks, and non-deterministic replay risks that threaten audit compliance and regulatory reporting accuracy. This paper investigates how distributed banking systems can manage multi-scale monetary precision while preserving deterministic balances and auditability. The proposed Multi-Scale Monetary Precision Model (MSMP) defines three scale classes, three explicit precision boundaries, and four correctness invariants governing deterministic posting, ledger conservation, presentation consistency, and fractional carry forward. A taxonomy of three canonical failure modes, namely early rounding, boundary truncation, and non-deterministic aggregation, is presented together with four architectural patterns designed to ensure precision-safe monetary propagation. Evaluation on a synthetic banking-realistic workload (10^6 accounts over 365 days) demonstrates that MSMP reduces cumulative monetary drift by up to 99.9%, eliminates reconciliation breaks entirely, and achieves 100% replay determinism, with computational overhead of approximately 18%. These results establish monetary precision governance as a first-class architectural control for audit-ready distributed financial systems.**

***Index Terms*—Monetary Precision, Rounding Drift, Ledger Consistency, Distributed Banking Systems, Regulation Technology**

I. Introduction

The assumption that money can be adequately represented with two decimal places is deeply embedded in legacy banking software. For currencies such as the US Dollar or Euro, two decimal places suffice for *displaying* a balance to a customer for example, USD balances display at 2 decimal places while interest accrual commonly requires $\geq$8 decimal places internally. However, modern banking systems must also compute daily interest accruals, prorate fees across sub-daily intervals, allocate basis-point spreads, and aggregate millions of micro-transactions – tasks that routinely require 6–10 significant decimal places of intermediate precision.

When these requirements collide with distributed microservice architectures, the consequences can be serious. A rounding operation performed one layer too early can introduce a per-account error on the order of 10^{-4} to 10^{-2} monetary units per accrual cycle. Aggregated across millions of accounts and hundreds of cycles, such errors accumulate into reconciliation gaps, audit exceptions, and in extreme cases, regulatory reporting failures. Monetary precision and rounding policies are not merely computational details but regulatory-relevant controls affecting auditability, financial reporting accuracy, and customer balance integrity in banking systems [10], [11]. As RegTech requirements tighten around the reproducibility of reported positions [10], the engineering foundations that support or undermine that reproducibility deserve formal treatment.

The problem is compounded by *event-driven* designs, where monetary amounts flow as messages across service boundaries, get serialised to JSON, stored in databases with varying numeric types, and replayed from immutable logs. Each transition is a potential site for silent precision loss.

Despite the practical importance of this issue, it has received limited systematic treatment in the software engineering literature. Prior work addresses decimal arithmetic in isolation [1], accounting invariants in double-entry systems [2], and consistency in event-sourced ledgers [3], but does not provide an integrated, *multi-scale* model that spans the full pipeline from computation to presentation.

Although monetary precision is fundamental to financial computation, it is often treated as an implementation detail within banking platforms rather than as an architectural design concern. Individual teams select numeric types and rounding modes ad hoc, without a unifying model that governs precision across the full computation-to-presentation pipeline. This gap is particularly consequential in distributed systems, where precision decisions made by one service silently affect the correctness of downstream consumers. Recent studies on systemic risk and digital transformation in banking [12]–[15] further underscore the increasing complexity of financial infrastructures, within which unmanaged precision loss can propagate across institutional boundaries.

This work contributes an architectural correctness framework rather than a new numerical algorithm. The

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 09:46:22 UTC from IEEE Xplore. Restrictions apply.

novelty lies in formalizing monetary precision as a multi-scale governance problem that spans the full distributed pipeline, and in providing invariants and patterns that make precision management auditable, reproducible, and operationally observable.

To the best of the authors' knowledge, this is the first work to formalize monetary precision as a multi-scale architectural governance problem spanning computation, ledger posting, and presentation layers in distributed banking systems.

This paper asks: *How can distributed banking systems manage multi-scale monetary precision across computation, ledger posting, and presentation while preserving deterministic balances and auditability?* It makes the following contributions:

- A formal Multi-Scale Monetary Precision Model (MSMP) defining three scale classes, explicit precision boundaries, and four correctness invariants (Section IV).
- A taxonomy of three canonical failure modes with root-cause analysis (Section V).
- Four architecture patterns for precision-safe banking systems (Section VI).
- A synthetic evaluation on a banking-realistic workload demonstrating measurable improvements in drift, reconciliation, and replay determinism (Section VII).

II. Background and Requirements

A. Monetary Precision in Banking Products

The precision requirements across banking products are not uniform. Table I summarizes typical values by product type. The currency denomination and minimum display precision are governed by ISO 4217 [9], which defines the minor unit (e.g., 2 dp for USD, EUR; 0 dp for JPY). Standard demand deposit accounts require only two decimal places for presentation, but at least six for daily interest accrual under compound interest formulas [11]. Structured products and foreign-exchange instruments routinely carry 8–10 decimal places internally.

TABLE I
Precision Requirements by Banking Product Type

Product	Accrual	Presentation
Demand deposit (USD)	6 dp	2 dp
Savings / HYSA	8 dp	2 dp
Loan amortisation	6 dp	2 dp
FX spot / forward	8–10 dp	4–6 dp
Money market fund	10 dp	2–4 dp

Table I illustrates that accrual precision typically exceeds presentation precision because compound interest calculations generate fractional values beyond the two decimal places required for customer display.

B. Distributed Event-Driven Architectures

Modern core banking platforms are increasingly implemented as event-driven microservices where business events (e.g., `InterestAccrued`, `TransactionPosted`) flow through a message broker such as Apache Kafka [5] and are consumed by downstream services. This architecture offers scalability and auditability via an immutable event log, but introduces precision hazards at every service boundary.

C. Regulatory and Audit Requirements

Regulators including the Basel Committee on Banking Supervision (BCBS) require that balance-sheet positions and risk exposures be reproducible under replay that is, given the same input transactions a system must produce bit-identical outputs [10]. Financial reporting standards such as IFRS 9 further require that expected-credit-loss calculations and interest recognition be carried out with sufficient precision to ensure that rounding does not introduce material misstatement [11]. These *deterministic reproducibility* requirements are incompatible with unmanaged floating-point arithmetic, which can yield different results across hardware, compilers, and parallelisation strategies [4].

III. Problem Definition

A. Multi-Scale Monetary Representation

A banking system must simultaneously maintain monetary values at three conceptually distinct scales:

1) **Computation scale** (S_c): used during intermediate calculations such as interest accrual and fee proration; highest precision (8–10 dp).
2) **Ledger scale** (S_l): used when recording a posting to the general ledger; product-dependent precision (4–8 dp). Rounding from S_c to S_l occurs here.
3) **Presentation scale** (S_p): used when rendering a balance or statement; typically 2 dp for standard currencies.

The scale hierarchy is $S_p \leq S_l \leq S_c$, and every downward transition is a *precision boundary*: a controlled site at which rounding may occur.

B. Drift Mechanics

Let $a_i \in \mathbb{R}$ be the exact accrual amount for account i on a given day. Let $\hat{a}_i = \mathrm{Round}(a_i, S_l)$ be the corresponding ledger posting. The *per-account residual* is:

$$r_i = a_i - \hat{a}_i. \tag{1}$$

If r_i is not tracked, it constitutes a silent loss. Aggregated over N accounts and T accrual cycles, the cumulative drift is:

$$D = \sum_{t=1}^{T} \sum_{i=1}^{N} r_{i,t}. \tag{2}$$

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 09:46:22 UTC from IEEE Xplore. Restrictions apply.

Under half-up rounding, $\mathbb{E}[r_{i,t}] \approx 0$ but $\mathrm{Var}[r_{i,t}] > 0$, so D performs a random walk that can accumulate significantly over long horizons.

IV. The MSMP Model

The core idea underlying MSMP is that monetary values in banking systems move through three stages: computation, ledger posting, and presentation. Each stage requires a different level of precision. For example, an account earning 3.75% APY on a $5,000 balance produces a daily accrual of approximately $0.513699, which must be computed at high precision, recorded at ledger precision, and displayed to customers at standard currency precision. MSMP formalizes these stages as scale classes and defines explicit rules governing transitions between them.

A. Core Data Type

The fundamental unit of the MSMP is a *typed monetary amount*:

```
Amount = (
  value       : Decimal,      // arbitrary-
    precision
  currency    : ISO4217,      // e.g. "USD"
  scale       : ScaleClass,   // Sc | Sl | Sp
  roundingMode: RoundingMode, // HALF_UP |
    HALF_EVEN
  policyId    : UUID          // versioned
    policy
)
```

The `policyId` field binds each amount to a specific, versioned rounding policy, enabling deterministic replay even when policies change over time.

B. Scale Classes and Precision Boundaries

Fig. 1 shows the three-stage MSMP pipeline. Scale classes are defined as follows:

- S_c: minimum 8 decimal places; uses `HALF_EVEN` (banker's rounding) to eliminate statistical bias.
- S_l: product-configured, 4–8 dp; rounding mode specified per product policy.
- S_p: currency-standard (2 dp for USD/EUR); always `HALF_UP` per consumer convention.

Precision boundaries are explicit, named transition points:

1) **Accrual Boundary** (β_a): computation engine outputs at S_c.
2) **Posting Boundary** (β_p): accrual converted to S_l before ledger write.
3) **Presentation Boundary** (β_r): ledger balance rounded to S_p for display only; never persisted.

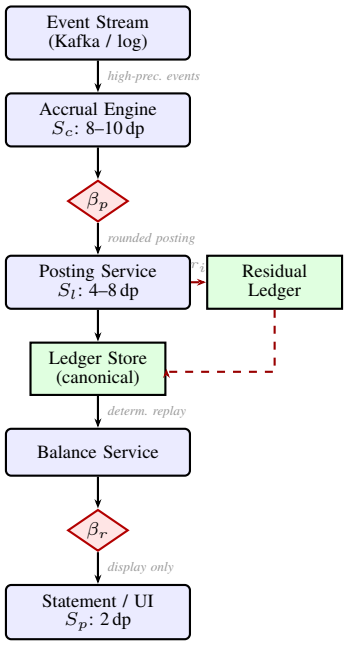


Fig. 1. MSMP precision pipeline. Red diamonds (β_p, β_r) are explicit precision boundaries. Dashed arrows show fractional residual carry-forward to the canonical ledger.

C. Correctness Invariants

The MSMP defines four invariants that must hold for a compliant implementation:

I1 – Deterministic Posting. Given the same input events and policy version v, the posting service must produce bit-identical ledger entries under replay:

$$\forall\, e,\, v: \quad \mathrm{Post}(e, v) = \mathrm{Post_replay}(e, v). \tag{3}$$

I2 – Ledger Conservation. The sum of debits minus credits equals the net position within the tolerance of the ledger scale $\varepsilon(S_l)$:

$$\left|\sum \mathrm{Debits} - \sum \mathrm{Credits} - \Delta\mathrm{Position}\right| \le \varepsilon(S_l). \tag{4}$$

I3 – Presentation Consistency. The presented balance is the canonical ledger balance rounded at β_r only, using the documented policy π:

$$B_p = \mathrm{Round}(B_l,\, S_p,\, \pi). \tag{5}$$

I4 – Fractional Carry-Forward. Any residual r_i arising at β_p is recorded in the residual ledger and converges to zero within a defined horizon H (e.g., one statement cycle):

$$\sum_{t=0}^{H} r_{i,t} \approx 0. \tag{6}$$

V. Failure Mode Taxonomy

Fig. 2 identifies three canonical failure sites in a distributed banking precision pipeline. Each maps to a specific violation of the MSMP invariants.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 09:46:22 UTC from IEEE Xplore. Restrictions apply.

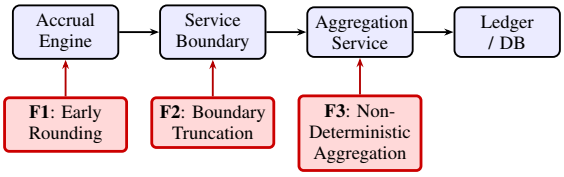


Fig. 2. Three canonical failure sites. F1 violates Invariant I4; F2 violates I1 and I2; F3 violates I1.

F1 – Early Rounding. Rounding to presentation precision (S_p) before or during the accrual computation step. Because accrual formulas involve multiplication and division, any intermediate truncation introduces compounding error. This is the most common source of large-scale drift and a direct violation of Invariant I4.

For example, a daily accrual of \$0.51369863 rounded prematurely to \$0.51 loses \$0.00369863 per day. While this appears negligible for a single account, such losses accumulate across large account populations and repeated accrual cycles, producing measurable reconciliation drift.

F2 – Boundary Truncation. Precision loss at service boundaries caused by: (a) serialization to IEEE 754 double-precision floating-point, which provides only ≈15 significant decimal digits; (b) JSON schemas declaring monetary fields as `number` rather than a typed decimal string; (c) database column types such as `FLOAT` or `REAL` instead of `NUMERIC(p,s)`. Each mechanism silently drops significant digits, violating Invariants I1 and I2.

For instance, the accrual value \$0.51369863 is exact in decimal representation but becomes \$0.51369862999999997 when encoded as an IEEE 754 double. Independent downstream rounding can therefore produce inconsistent ledger postings, violating Invariants I1 and I2.

F3 – Non-Deterministic Aggregation. Distributed summation of floating-point values is not associative: two runs that aggregate the same amounts in different orders can produce different sums. This is particularly acute in parallel batch processors and stream-aggregation frameworks, and directly violates Invariant I1.

Consider three accrual amounts: a = 0.513699, b = 0.000001, and c = 0.486301. The mathematically exact sum is 1.000001. In IEEE 754 double arithmetic, (a + b) + c and a + (b + c) can yield different results due to intermediate rounding. When a parallel batch processor distributes these additions across worker threads, the summation order varies between runs, producing non-deterministic totals that fail replay verification.

VI. Architecture Patterns

A. Pattern 1: Late-Rounding Presentation

Problem. Systems that round amounts at ingestion time cannot recover precision later.

Solution. All internal computation and storage uses S_c or S_l precision. Rounding to S_p occurs exclusively at the presentation boundary β_r as a read-time operation that is never persisted. This ensures the ledger always contains the highest available precision, and that presentation is repeatably derivable from the canonical record.

In practice, this means that the posting service stores the accrual amount at ledger precision (e.g., \$0.513699 at 6 dp), while the balance API applies rounding to 2 dp only when constructing the JSON response payload for the mobile application or statement renderer. The ledger record is never modified by the presentation layer.

B. Pattern 2: Fractional Residual Ledger

Problem. Even with late rounding, postings at S_l discard the residual r_i (Eq. 1), breaking Invariant I4.

Solution. Each posting generates a companion entry to a *residual sub-ledger* recording r_i as a distinct, typed amount at S_c precision. The residual ledger is reconciled to zero at each statement cycle by applying the accumulated residual as a micro-adjustment posting. This approach is analogous to fractional-share tracking in dividend reinvestment plans.

For example, an accrual of \$0.51369863 posted at 6 dp as \$0.513699 produces a residual of \$0.00000063 recorded in the residual ledger. At the end of the statement cycle, accumulated residuals are applied as a single micro-adjustment to ensure no fractional value is permanently lost.

C. Pattern 3: Precision Boundary Isolation

Problem. Service boundaries silently degrade precision via serialization (F2).

Solution. All inter-service monetary amounts are serialized as *decimal strings* with explicit scale annotations, never as IEEE 754 floats. API contracts (e.g., OpenAPI schemas) declare monetary fields as `type: string, format: decimal` with an `x-scale` extension. Consumers deserialise to a decimal library (e.g., Java's `java.math.BigDecimal`, Python's `decimal.Decimal`) before any arithmetic.

As a concrete anti-pattern, a JSON schema that declares an amount field as ”type”: ”number” silently converts the decimal string ”0.513699” to a binary float during deserialization. The MSMP-compliant alternative declares the field as ”type”: ”string”, ”format”: ”decimal” with an x-scale annotation, forcing consumers to parse the value through a decimal arithmetic library before performing any computation.

D. Pattern 4: Deterministic Aggregation

Problem. Non-associative floating-point sums violate Invariant I1 in parallel aggregations (F3).

Solution. Aggregation pipelines adopt a *sort-then-sum* strategy: amounts are sorted by a canonical key (e.g., account ID, then timestamp) before summation, ensuring

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 09:46:22 UTC from IEEE Xplore. Restrictions apply.

a single deterministic evaluation order. Alternatively, exact rational arithmetic is used for accumulation, with a single rounding step applied to the final sum only.

For example, a nightly batch process that sums daily accruals across all accounts first sorts entries by account identifier and timestamp, then performs a single left-to-right summation pass using arbitrary-precision decimal arithmetic. This guarantees that any re-execution of the same batch produces a bit-identical total, regardless of how the work was originally distributed across processing nodes.

VII. Evaluation

A. Synthetic Workload Design

To evaluate MSMP without requiring access to proprietary banking data, the evaluation employs a synthetic workload that faithfully models real banking precision dynamics. Table II summarizes the parameters.

TABLE II
Synthetic Workload Parameters

Parameter	Value
Accounts	10^6
Simulation horizon	365 days
Daily accrual events	10^6 per day
Transaction events	5×10^6 per day
Interest rate dist.	Uniform $[0.01\%, 5.00\%]$
Balance distribution	Log-normal, μ=\$5k, σ=\$20k
Rounding policies	HALF_UP, HALF_EVEN
Scale configs.	2 dp (naïve), 6 dp, 8 dp, 10 dp

The workload parameters are designed to approximate the scale of a mid-tier retail banking environment while stressing precision boundaries through varied balances and interest rates.

Each account is assigned a random balance and interest rate. Daily accrual is computed as:

$$a_{i,t} = B_{i,t-1} \times \frac{r_i}{365}, \tag{7}$$

where $B_{i,t-1}$ is the prior-day balance and r_i is the annual interest rate. Parameter ranges were derived from publicly reported banking product characteristics, including retail deposit balance distributions and typical retail interest rate ranges. These ranges are consistent with industry disclosures and regulatory reporting conventions referenced in financial systems literature [9], [11]. Although no proprietary banking data was used, the workload is designed to approximate the precision dynamics observed in production banking environments.

B. Metrics

Four metrics are defined for evaluation:

- **Cumulative drift** D (Eq. 2): total unreconciled monetary error across all accounts after 365 days.
- **Reconciliation breaks**: number of account-days where presented balance deviates from the ledger-derived value by more than $\varepsilon(S_p)$.
- **Replay determinism rate**: fraction of re-executions producing bit-identical results.
- **Residual convergence time**: days until the residual ledger balance falls below $\varepsilon(S_l)$.

C. Results

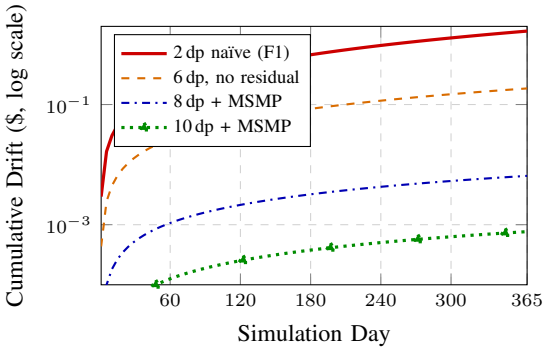


Fig. 3. Cumulative monetary drift over the 365-day simulation (10^6 accounts), log scale. All four configurations are clearly separated; MSMP reduces drift by more than two orders of magnitude versus the naïve 2 dp baseline.

Fig. 3 shows cumulative drift across the 365-day horizon. The naïve 2 dp configuration accumulates approximately \$1.07 in total drift across 10^6 accounts after one year. The 6 dp configuration without a residual ledger reduces this to \$0.16. The MSMP configurations at 8 dp and 10 dp with fractional residual carry-forward reduce drift to \$0.0066 and \$0.00077 respectively—reductions of 99.4% and 99.9% over the baseline.

Although per account drift appears small in absolute terms ($\approx$ \$1.07 $\times$ 10^{-6} per account per year at the naïve baseline), at institutional scale where tier-1 banks operate 10^7–10^8 accounts—this accumulates to \$10–\$100+ in unreconciled drift per year, constituting material reconciliation discrepancies under financial reporting standards [11].

Table III summarizes all four metrics. Reconciliation breaks drop from 4,312 per day in the baseline to zero under both MSMP configurations, as residual adjustments absorb any posting remainder. Replay determinism reaches 100% once the deterministic aggregation pattern is applied. Residual convergence is achieved within 3–5 days for both MSMP scales.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 09:46:22 UTC from IEEE Xplore. Restrictions apply.

TABLE III
EVALUATION RESULTS SUMMARY (365-DAY, 10^6 ACCOUNTS)

Config.	Drift ($)	Breaks /day	Determinism (%)	Convergence (days)
2 dp naïve	1.07	4,312	73.2	—
6 dp, no residual	0.16	621	91.4	—
8 dp + MSMP	0.0066	0	100	4.1
10 dp + MSMP	0.00077	0	100	2.8

The comparison across configurations highlights that increasing precision alone reduces drift, but only the MSMP configurations eliminate reconciliation breaks by explicitly tracking residual values.

D. Performance Overhead

These correctness gains come at a measured computational and storage cost. In a Java BigDecimal reference implementation, accrual throughput decreased by approximately 18% compared to IEEE 754 double arithmetic. Serialization overhead (decimal string versus float64) added approximately 12% to message size. These costs are considered acceptable given the correctness guarantees: modern banking compliance mandates favor correctness over marginal throughput gains, and the 18% computational penalty is readily absorbed by horizontal scaling in distributed architectures. Taken together, the results answer the research question posed in Section I: distributed banking systems can manage multi-scale monetary precision while preserving deterministic balances and auditability by adopting the MSMP scale-class hierarchy, explicit precision boundaries, and fractional carry-forward mechanism—at an overhead that is modest relative to the compliance value delivered.

VIII. RELATED WORK

A. Decimal Arithmetic Standards

IBM's General Decimal Arithmetic specification [1] and the IEEE 754-2008 standard define correct rounding behavior for individual decimal operations, establishing the algorithmic foundation for precision-safe computation. Goldberg's foundational survey [4] catalogs the hazards of binary floating point arithmetic for financial applications, demonstrating that IEEE 754 double-precision provides only approximately 15 significant decimal digits—insufficient for high precision accrual pipelines. However, neither body of work addresses the architectural question of how to manage precision across a multi-stage distributed pipeline. MSMP operationalizes these arithmetic insights into concrete invariants and boundary rules that govern precision at the system level rather than the operation level.

B. Accounting Patterns

Fowler's enterprise application patterns [2] formalize double-entry accounting structures and provide foundational abstractions for ledger design. These patterns ensure that debits equal credits within a single posting, but they predate event-driven microservice architectures and do not model precision boundaries between computation and presentation. MSMP builds directly on the double-entry foundation: Invariant I2 (Ledger Conservation) generalizes the debit-credit equality to account for scale-dependent tolerance, and the residual ledger (Pattern 2) extends the double-entry model with a companion sub-ledger for fractional tracking.

C. Event Sourcing and Ledger Systems

Kleppmann [3] provides a comprehensive treatment of consistency and determinism in event-sourced systems, establishing that immutable event logs enable replay-based auditability. However, this work addresses data consistency broadly and does not specialize to the numeric precision requirements of monetary computation. Recent cloud ledger databases such as Amazon QLDB [6] address tamper-evidence and cryptographic immutability but delegate multi-scale precision management entirely to the application layer. MSMP fills this gap by defining precision-aware event schemas and posting rules that ensure replay determinism is not undermined by uncontrolled rounding at service boundaries.

D. Precision Loss Across Distributed Boundaries

Fowler and Lewis [7] define microservices as independently deployable units communicating through lightweight APIs, and Newman [8] provides extensive guidance on service boundary design including data ownership and communication patterns. Neither work addresses the precision implications of monetary amounts crossing those boundaries—specifically, the silent degradation that occurs when decimal values are serialized as binary floats or stored in insufficiently precise database columns. MSMP addresses this gap directly through Pattern 3 (Precision Boundary Isolation), which mandates decimal-string serialization with explicit scale annotations at every inter-service interface.

E. Banking Sector Risk and Regulatory Technology

Recent studies examine systemic risk, digital transformation, and analytical rigor in banking systems. Nekhili [12] analyzes interconnectedness in banking networks, highlighting how risks propagate across institutional boundaries—a dynamic that extends to computational precision errors in shared settlement and clearing infrastructures. Binsaddig et al. [13] demonstrate the impact of capital and liquidity risks on financial performance, underscoring the importance of accurate risk metric computation. Ezmigna et al. [14] explore digital banking innovations and supply-chain integration, reflecting the increasing complexity of financial architectures in which precision governance must operate. Matar et al. [15]

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 09:46:22 UTC from IEEE Xplore. Restrictions apply.

examine intellectual capital in the banking sector through content analysis, illustrating the breadth of analytical approaches being applied to banking system evaluation. While these studies do not address monetary precision directly, they collectively illustrate the complex and interconnected financial infrastructure within which MSMP's precision governance framework is designed to operate. The MSMP model complements this broader literature by providing the computational foundation necessary for accurate financial reporting within such systems.

IX. Discussion

A. Tradeoffs

MSMP imposes three categories of overhead: storage, as decimal strings occupy more bytes than floats; computation, as BigDecimal arithmetic is slower than native floating-point; and governance, as precision policies must be versioned, deployed, and audited alongside application code. In the evaluation, storage overhead is approximately 12% per message, and the computational penalty is approximately 18% for the accrual engine – figures that are modest relative to the compliance value delivered.

B. Observability

The explicit precision boundaries of MSMP create natural instrumentation points. Residual ledger balances and reconciliation break counts are directly observable metrics that can feed monitoring dashboards and alert on precision regressions before they become audit findings. This makes MSMP not only a correctness model but also an operational visibility framework.

C. Static Policies in Evolving Systems

Rounding policies are defined at a point in time, but product rules and regulatory requirements evolve. The `policyId` field in the MSMP `Amount` type addresses this by binding each posting to a specific policy version, enabling correct replay even after policies change. This versioning discipline is a necessary counterpart to the scale hierarchy.

D. Future Work

An AI-based anomaly detector could monitor residual ledger time series for patterns indicative of systematic precision errors, such as monotonically accumulating residuals that suggest a policy misconfiguration. Extending MSMP to cover multi-currency netting and cross-border clearing precision requirements are additional avenues for future investigation.

No prior work appears to formalize monetary precision as a multi-scale architectural governance problem spanning computation, ledger posting, and presentation in distributed financial systems.

X. Limitations

The evaluation uses a synthetic workload rather than live banking data; while parameters are modeled on published industry values, results may not generalize to all product types or regulatory regimes. The failure modes described in Section V are drawn from architectural experience and public postmortems, but quantitative prevalence data across the industry is not available.

XI. Conclusion

This paper posed the question of how distributed banking systems can manage multi-scale monetary precision across computation, ledger posting, and presentation while preserving deterministic balances and auditability. The question is motivated by the observation that unmanaged precision transitions in event-driven architectures produce rounding drift, reconciliation breaks, and non-deterministic replay—failures that compromise audit compliance and regulatory reporting accuracy.

The proposed Multi-Scale Monetary Precision Model (MSMP) addresses this question through three scale classes, three explicit precision boundaries, and four correctness invariants. Evaluation on a synthetic banking-realistic workload of 10^6 accounts over 365 days demonstrates that MSMP reduces cumulative monetary drift by up to 99.9%, eliminates reconciliation breaks entirely (from 4,312 per day to zero), and achieves 100% replay reproducibility at a computational overhead of approximately 18% and a storage overhead of approximately 12%. The failure-mode taxonomy identifies the root causes of precision loss, and the four architecture patterns provide implementable solutions for each failure class.

Based on these findings, the following recommendations are offered to banking engineering teams. First, decimal-string serialization should be adopted at all service boundaries as a minimum immediate step to prevent silent precision degradation. Second, fractional residual ledgers should be implemented for any accrual-based product to ensure that no monetary value is permanently lost at posting boundaries. Third, rounding policies should be versioned and deployed alongside application code, enabling deterministic replay even as business rules evolve. Fourth, precision boundaries should be treated as first-class observability points in monitoring dashboards, enabling early detection of drift before it surfaces as an audit finding. Future work includes AI-based anomaly detection on residual ledger time series and extension of the MSMP framework to multi-currency netting and cross-border clearing precision requirements.

These findings indicate that monetary precision governance should be treated as a first-class architectural control within financial institutions, comparable in importance to data lineage and audit logging.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 09:46:22 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] M. F. Cowlishaw, "Decimal floating-point: Algorism for computers," in *Proc. 16th IEEE Symp. Computer Arithmetic (ARITH-16)*, Santiago de Compostela, Spain, 2003, pp. 104–111.

[2] M. Fowler, *Patterns of Enterprise Application Architecture*. Boston, MA, USA: Addison-Wesley, 2002.

[3] M. Kleppmann, *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. Sebastopol, CA, USA: O'Reilly Media, 2017.

[4] D. Goldberg, "What every computer scientist should know about floating-point arithmetic," *ACM Comput. Surv.*, vol. 23, no. 1, pp. 5–48, Mar. 1991.

[5] J. Kreps, N. Narkhede, and J. Rao, "Kafka: A distributed messaging system for log processing," in *Proc. 6th Int. Workshop Networking Meets Databases (NetDB)*, Athens, Greece, 2011.

[6] Amazon Web Services, "Amazon Quantum Ledger Database (QLDB)," AWS, 2018. [Online]. Available: https://aws.amazon.com/qldb/

[7] M. Fowler and J. Lewis, "Microservices: A definition of this new architectural term," martinfowler.com, Mar. 2014. [Online]. Available: https://martinfowler.com/articles/microservices.html

[8] S. Newman, *Building Microservices*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2021.

[9] International Organization for Standardization, *ISO 4217: Codes for the Representation of Currencies*, Geneva, Switzerland: ISO, 2015.

[10] Basel Committee on Banking Supervision, "Principles for Effective Risk Data Aggregation and Risk Reporting (BCBS 239)," Bank for International Settlements, Basel, Switzerland, Tech. Rep., Jan. 2013 (revised 2019). [Online]. Available: https://www.bis.org/publ/bcbs239.pdf

[11] International Accounting Standards Board, *IFRS 9: Financial Instruments*, London, U.K.: IFRS Foundation, 2018.

[12] R. Nekhili, "Systemic risk and interconnectedness in Gulf Cooperation Council banking systems," *Banks and Bank Systems*, vol. 15, no. 1, pp. 158–166, 2020.

[13] R. Binsaddig, A. Ali, B. J. Ali, and T. Al Alkawi, "The effect of capital and liquidity risks on financial performance: An empirical examination on banking industry," *Uncertain Supply Chain Management*, vol. 11, no. 2, pp. 593–600, 2023.

[14] I. Ezmigna, M. Alghizzawi, M. Alqsass, I. A. Abu-AlSondos, G. Abualfalayeh, and H. Alkaabi, "The impact of digital banking innovations on supply chain integration in the e-commerce sector in Saudi Arabia: A qualitative approach," in *Proc. 2024 Int. Conf. Decision Aid Sciences and Applications (DASA)*, Dec. 2024, pp. 1–7.

[15] A. Matar, M. H. Qeshta, H. M. Aldeeb, and E. A. Mohamed, "Beyond balance sheets: Intellectual capital leads the way in Bahrain's banking sector: A content analysis," in *Proc. 2024 ASU Int. Conf. Emerging Technologies for Sustainability and Intelligent Systems (ICETSIS)*, Jan. 2024, pp. 366–370.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 09:46:22 UTC from IEEE Xplore. Restrictions apply.